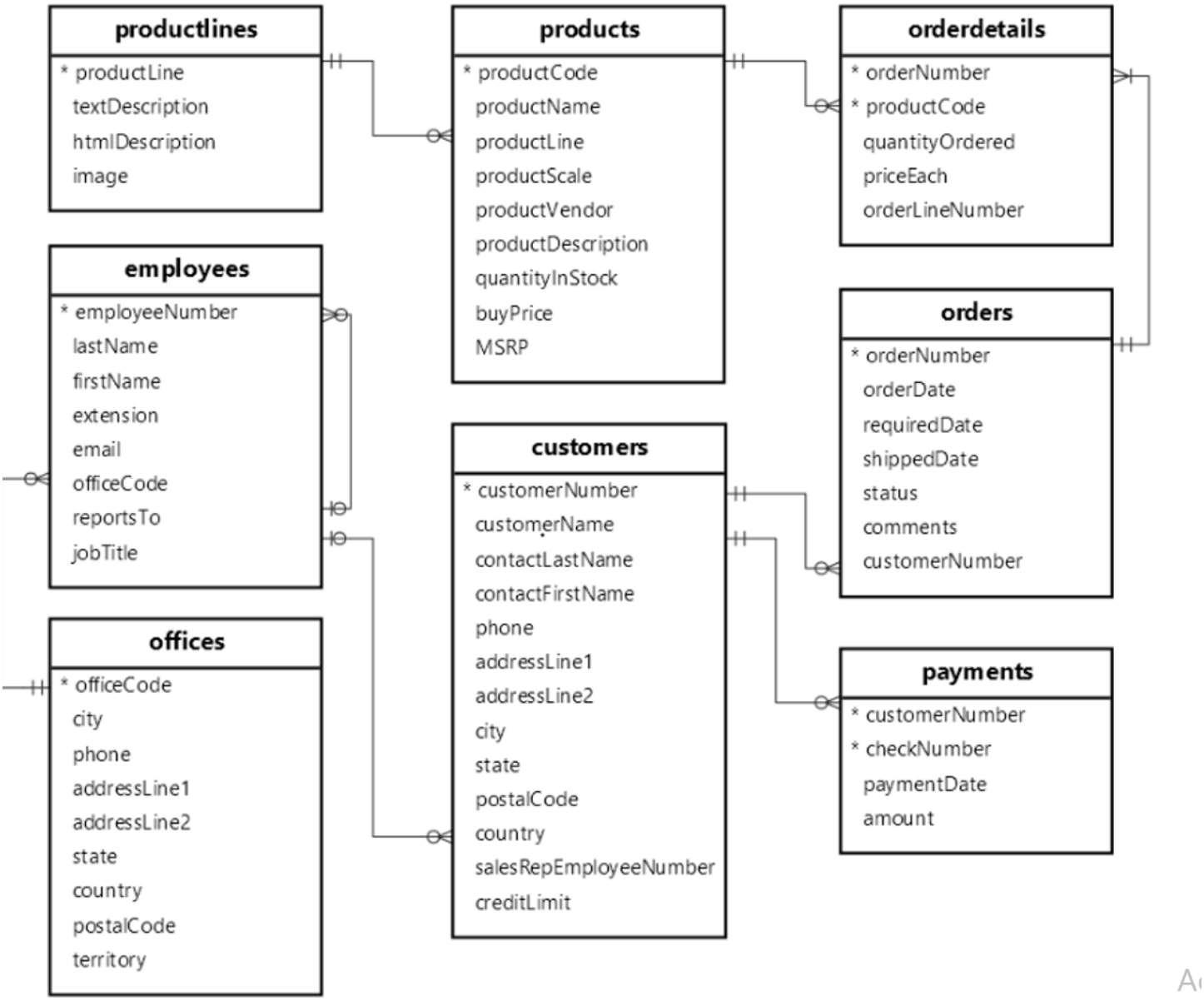

The fields above mentioned in the * are the Primary keys of that table
eg: *officeCode, *CustomerNumber etc.

. **CREATE TABLES using the below structure and insert data into it**

```
-- OFFICES
CREATE TABLE offices (
   officeCode VARCHAR(10) PRIMARY KEY,
   city VARCHAR(50),
   phone VARCHAR(50),
   addressLine1 VARCHAR(100),
   addressLine2 VARCHAR(100),
   state VARCHAR(50),
   country VARCHAR(50),
   postalCode VARCHAR(15),
```

```
    territory VARCHAR(50)
);

-- EMPLOYEES
CREATE TABLE employees (
    employeeNumber INT PRIMARY KEY,
    lastName VARCHAR(50),
    firstName VARCHAR(50),
    extension VARCHAR(10),
    email VARCHAR(100),
    officeCode VARCHAR(10),
    reportsTo INT,
    jobTitle VARCHAR(50),
    FOREIGN KEY (officeCode) REFERENCES offices(officeCode),
    FOREIGN KEY (reportsTo) REFERENCES employees(employeeNumber)
);

-- CUSTOMERS
CREATE TABLE customers (
    customerNumber INT PRIMARY KEY,
    customerName VARCHAR(100),
    contactLastName VARCHAR(50),
    contactFirstName VARCHAR(50),
    phone VARCHAR(50),
    addressLine1 VARCHAR(100),
    addressLine2 VARCHAR(100),
    city VARCHAR(50),
    state VARCHAR(50),
    postalCode VARCHAR(15),
    country VARCHAR(50),
    salesRepEmployeeNumber INT,
    creditLimit DECIMAL(10,2),
    FOREIGN KEY (salesRepEmployeeNumber) REFERENCES employees(employeeNumber)
);

-- PRODUCTLINES
CREATE TABLE productlines (
    productLine VARCHAR(50) PRIMARY KEY,
    textDescription TEXT,
    htmlDescription TEXT,
    image TEXT
);

-- PRODUCTS
CREATE TABLE products (
    productCode VARCHAR(15) PRIMARY KEY,
```

```
    productName VARCHAR(100),
    productLine VARCHAR(50),
    productScale VARCHAR(10),
    productVendor VARCHAR(50),
    productDescription TEXT,
    quantityInStock INT,
    buyPrice DECIMAL(10,2),
    MSRP DECIMAL(10,2),
    FOREIGN KEY (productLine) REFERENCES productlines(productLine)
);

-- ORDERS
CREATE TABLE orders (
    orderNumber INT PRIMARY KEY,
    orderDate DATE,
    requiredDate DATE,
    shippedDate DATE,
    status VARCHAR(20),
    comments TEXT,
    customerNumber INT,
    FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
);

-- ORDERDETAILS
CREATE TABLE orderdetails (
    orderNumber INT,
    productCode VARCHAR(15),
    quantityOrdered INT,
    priceEach DECIMAL(10,2),
    orderLineNumber INT,
    PRIMARY KEY (orderNumber, productCode),
    FOREIGN KEY (orderNumber) REFERENCES orders(orderNumber),
    FOREIGN KEY (productCode) REFERENCES products(productCode)
);

-- PAYMENTS
CREATE TABLE payments (
    customerNumber INT,
    checkNumber VARCHAR(50),
    paymentDate DATE,
    amount DECIMAL(10,2),
    PRIMARY KEY (customerNumber, checkNumber),
    FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
);
```

INSERT SAMPLE DATA

Offices

```
INSERT INTO offices VALUES
('1', 'New York', '1234567890', '5th Ave', NULL, 'NY', 'USA', '10001', 'NA'),
('2', 'London', '9876543210', 'Baker St', NULL, NULL, 'UK', 'NW1', 'EMEA');
```

Employees

```
INSERT INTO employees VALUES
(1001, 'Smith', 'John', 'x101', 'john@company.com', '1', NULL, 'Manager'),
(1002, 'Doe', 'Jane', 'x102', 'jane@company.com', '1', 1001, 'Sales Rep'),
(1003, 'Brown', 'Mike', 'x103', 'mike@company.com', '2', 1001, 'Sales Rep');
```

Customers

```
INSERT INTO customers VALUES
(2001, 'ABC Corp', 'Lee', 'Chris', '1111111111', 'Street 1', NULL, 'NYC', 'NY', '10001', 'USA', 1002, 50000),
(2002, 'XYZ Ltd', 'Kim', 'Pat', '2222222222', 'Street 2', NULL, 'London', NULL, 'NW1', 'UK', 1003, 70000),
(2003, 'Global Inc', 'Singh', 'Raj', '3333333333', 'Street 3', NULL, 'Delhi', NULL, '110001', 'India', 1002, 60000);
```

Product Lines

```
INSERT INTO productlines VALUES
('Cars', 'Car models', NULL, NULL),
('Bikes', 'Bike models', NULL, NULL);
```

Products

```
INSERT INTO products VALUES
('P101', 'Toyota Camry', 'Cars', '1:18', 'Toyota', 'Sedan model', 100, 20.00, 30.00),
('P102', 'Honda Civic', 'Cars', '1:18', 'Honda', 'Compact model', 150, 18.00, 28.00),
('P201', 'Yamaha Bike', 'Bikes', '1:12', 'Yamaha', 'Sport bike', 200, 10.00, 15.00);
```

Orders

```
INSERT INTO orders VALUES
(3001, '2024-01-01', '2024-01-05', '2024-01-03', 'Shipped', NULL, 2001),
(3002, '2024-02-01', '2024-02-05', '2024-02-04', 'Shipped', NULL, 2002),
(3003, '2024-03-01', '2024-03-06', NULL, 'Pending', NULL, 2003);
```

Order Details

```
INSERT INTO orderdetails VALUES
(3001, 'P101', 10, 30.00, 1),
```

(3001, 'P201', 5, 15.00, 2),
(3002, 'P102', 7, 28.00, 1),
(3003, 'P101', 3, 30.00, 1);

Payments

INSERT INTO payments VALUES
(2001, 'CHK001', '2024-01-10', 300.00),
(2002, 'CHK002', '2024-02-10', 500.00),
(2003, 'CHK003', '2024-03-10', 150.00);

EASY LEVEL (1–20)

1. List all customers ordered by customerName.

```
mysql> SELECT * FROM customers ORDER BY customerName;
+----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
| customerNumber | customerName | contactLastName | contactFirstName | phone      | addressLine1 | addressLine2 | city   | state | postalCode | country | sa
lesRepEmployeeNumber | creditLimit |
+----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
|           2001 | ABC Corp     | Lee             | Chris            | 1111111111 | Street 1     | NULL         | NYC    | NY    | 10001      | USA     |
                1002 |    50000.00 |
|           2003 | Global Inc   | Singh           | Raj              | 3333333333 | Street 3     | NULL         | Delhi  | NULL  | 110001     | India   |
                1002 |    60000.00 |
|           2002 | XYZ Ltd      | Kim             | Pat              | 2222222222 | Street 2     | NULL         | London | NULL  | NW1        | UK      |
                1003 |    70000.00 |
+----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
3 rows in set (0.01 sec)
```

2. Display all products ordered by buyPrice descending.

```
mysql> SELECT * FROM products ORDER BY buyPrice DESC;
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
| productCode | productName  | productLine | productScale | productVendor | productDescription | quantityInStock | buyPrice | MSRP  |
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
| P101        | Toyota Camry | Cars        | 1:18         | Toyota        | Sedan model        |             100 |    20.00 | 30.00 |
| P102        | Honda Civic  | Cars        | 1:18         | Honda         | Compact model      |             150 |    18.00 | 28.00 |
| P201        | Yamaha Bike  | Bikes       | 1:12         | Yamaha        | Sport bike         |             200 |    10.00 | 15.00 |
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
3 rows in set (0.00 sec)
```

3. Show all employees ordered by lastName.

```
mysql> SELECT * FROM employees ORDER BY lastName;
+----------------+-----------+------------+------------+------------------+------------+------------+------------+
| employeeNumber | lastName  | firstName  | extension  | email            | officeCode | reportsTo  | jobTitle   |
+----------------+-----------+------------+------------+------------------+------------+------------+------------+
|           1003 | Brown     | Mike       | x103       | mike@company.com | 2          |       1001 | Sales Rep  |
|           1002 | Doe       | Jane       | x102       | jane@company.com | 1          |       1001 | Sales Rep  |
|           1001 | Smith     | John       | x101       | john@company.com | 1          |       NULL | Manager    |
+----------------+-----------+------------+------------+------------------+------------+------------+------------+
3 rows in set (0.00 sec)
```

4. List all offices sorted by country.

```
mysql> SELECT * FROM offices ORDER BY country;
+------------+----------+------------+--------------+--------------+-------+---------+------------+------------+
| officeCode | city     | phone      | addressLine1 | addressLine2 | state | country | postalCode | territory  |
+------------+----------+------------+--------------+--------------+-------+---------+------------+------------+
| 2          | London   | 9876543210 | Baker St     | NULL         | NULL  | UK      | NW1        | EMEA       |
| 1          | New York | 1234567890 | 5th Ave      | NULL         | NY    | USA     | 10001      | NA         |
+------------+----------+------------+--------------+--------------+-------+---------+------------+------------+
2 rows in set (0.00 sec)
```

5. Get all orders sorted by orderDate descending.

```
mysql> SELECT * FROM orders ORDER BY orderDate DESC;
+-------------+------------+--------------+-------------+---------+----------+----------------+
| orderNumber | orderDate  | requiredDate | shippedDate | status  | comments | customerNumber |
+-------------+------------+--------------+-------------+---------+----------+----------------+
|        3003 | 2024-03-01 | 2024-03-06   | NULL        | Pending | NULL     |           2003 |
|        3002 | 2024-02-01 | 2024-02-05   | 2024-02-04  | Shipped | NULL     |           2002 |
|        3001 | 2024-01-01 | 2024-01-05   | 2024-01-03  | Shipped | NULL     |           2001 |
+-------------+------------+--------------+-------------+---------+----------+----------------+
3 rows in set (0.00 sec)
```

6. Show customer names with their sales representative (employee name).

```
mysql> SELECT c.customerName, e.firstName, e.lastName
    -> FROM customers c
    -> JOIN employees e ON c.salesRepEmployeeNumber = e.employeeNumber;
+--------------+-----------+----------+
| customerName | firstName | lastName |
+--------------+-----------+----------+
| ABC Corp     | Jane      | Doe      |
| Global Inc   | Jane      | Doe      |
| XYZ Ltd      | Mike      | Brown    |
+--------------+-----------+----------+
3 rows in set (0.00 sec)
```

7. List all products with their product line.

```
mysql> SELECT p.productName, p.productLine FROM products p;
+--------------+-------------+
| productName  | productLine |
+--------------+-------------+
| Toyota Camry | Cars        |
| Honda Civic  | Cars        |
| Yamaha Bike  | Bikes       |
+--------------+-------------+
3 rows in set (0.00 sec)
```

8. Display orders with customer names.

```
mysql> SELECT o.orderNumber, c.customerName
    -> FROM orders o
    -> JOIN customers c ON o.customerNumber = c.customerNumber;
+-------------+--------------+
| orderNumber | customerName |
+-------------+--------------+
|        3001 | ABC Corp     |
|        3002 | XYZ Ltd      |
|        3003 | Global Inc   |
+-------------+--------------+
3 rows in set (0.00 sec)
```

9. Show order details with product names.

```
mysql> SELECT od.orderNumber, p.productName, od.quantityOrdered
    -> FROM orderdetails od
    -> JOIN products p ON od.productCode = p.productCode;
+-------------+--------------+-----------------+
| orderNumber | productName  | quantityOrdered |
+-------------+--------------+-----------------+
|        3001 | Toyota Camry |              10 |
|        3001 | Yamaha Bike  |               5 |
|        3002 | Honda Civic  |               7 |
|        3003 | Toyota Camry |               3 |
+-------------+--------------+-----------------+
4 rows in set (0.00 sec)
```

10. List payments with customer names.

```
mysql> SELECT p.checkNumber, c.customerName, p.amount
    -> FROM payments p
    -> JOIN customers c ON p.customerNumber = c.customerNumber;
+-------------+--------------+--------+
| checkNumber | customerName | amount |
+-------------+--------------+--------+
| CHK001      | ABC Corp     | 300.00 |
| CHK002      | XYZ Ltd      | 500.00 |
| CHK003      | Global Inc   | 150.00 |
+-------------+--------------+--------+
3 rows in set (0.00 sec)
```

11. Count total number of customers.

```
mysql> SELECT COUNT(*) FROM customers;
+----------+
| COUNT(*) |
+----------+
|        3 |
+----------+
1 row in set (0.00 sec)
```

12. Count total number of orders.

```
mysql> SELECT COUNT(*) FROM orders;
+----------+
| COUNT(*) |
+----------+
|        3 |
+----------+
1 row in set (0.00 sec)
```

13. Find average buyPrice of products.

```
mysql> SELECT AVG(buyPrice) FROM products;
+---------------+
| AVG(buyPrice) |
+---------------+
|     16.000000 |
+---------------+
1 row in set (0.00 sec)
```

14. Find maximum MSRP from products.

```
mysql> SELECT MAX(MSRP) FROM products;
+-----------+
| MAX(MSRP) |
+-----------+
|     30.00 |
+-----------+
1 row in set (0.00 sec)
```

15. Find total number of employees.

```
mysql> SELECT COUNT(*) FROM employees;
+----------+
| COUNT(*) |
+----------+
|        3 |
+----------+
```

16. Count customers per country.

```
mysql> SELECT country, COUNT(*) FROM customers GROUP BY country;
+---------+----------+
| country | COUNT(*) |
+---------+----------+
| India   |        1 |
| UK      |        1 |
| USA     |        1 |
+---------+----------+
3 rows in set (0.00 sec)
```

17. Count orders per status.

```
mysql> SELECT status, COUNT(*) FROM orders GROUP BY status;
+---------+----------+
| status  | COUNT(*) |
+---------+----------+
| Pending |        1 |
| Shipped |        2 |
+---------+----------+
2 rows in set (0.00 sec)
```

18. Count products per product line.

```
mysql> SELECT productLine, COUNT(*) FROM products GROUP BY productLine;
+-------------+----------+
| productLine | COUNT(*) |
+-------------+----------+
| Bikes       |        1 |
| Cars        |        2 |
+-------------+----------+
2 rows in set (0.00 sec)
```

19. Find average credit limit per country.

```
mysql> SELECT country, AVG(creditLimit) FROM customers GROUP BY country;
+---------+------------------+
| country | AVG(creditLimit) |
+---------+------------------+
| India   |     60000.000000 |
| UK      |     70000.000000 |
| USA     |     50000.000000 |
+---------+------------------+
3 rows in set (0.00 sec)
```

20. Count employees per office.

```
mysql> SELECT officeCode, COUNT(*) FROM employees GROUP BY officeCode;
+------------+----------+
| officeCode | COUNT(*) |
+------------+----------+
| 1          |        2 |
| 2          |        1 |
+------------+----------+
2 rows in set (0.00 sec)
```

MEDIUM LEVEL (21–50)

21. Count number of orders per customer.

```
ysql> SELECT customerNumber, COUNT(*) FROM orders GROUP BY customerNumber;
----------------+----------+
 customerNumber | COUNT(*) |
----------------+----------+
           2001 |        1 |
           2002 |        1 |
           2003 |        1 |
----------------+----------+
 rows in set (0.00 sec)
```

22. Find total payment amount per customer.

```
mysql> SELECT customerNumber, COUNT(*) FROM orders GROUP BY customerNumber;
+----------------+----------+
| customerNumber | COUNT(*) |
+----------------+----------+
|           2001 |        1 |
|           2002 |        1 |
|           2003 |        1 |
+----------------+----------+
3 rows in set (0.00 sec)
```

23. List total quantity ordered per product.

```
mysql> SELECT productCode, SUM(quantityOrdered) FROM orderdetai
roductCode;
+-------------+----------------------+
| productCode | SUM(quantityOrdered) |
+-------------+----------------------+
| P101        |                   13 |
| P102        |                    7 |
| P201        |                    5 |
+-------------+----------------------+
3 rows in set (0.00 sec)
```

24. Show total revenue per product (quantityOrdered * priceEach).

```
mysql> SELECT productCode, SUM(quantityOrdered * priceEach) AS revenue
    -> FROM orderdetails GROUP BY productCode;
+-------------+---------+
| productCode | revenue |
+-------------+---------+
| P101        |  390.00 |
| P102        |  196.00 |
| P201        |   75.00 |
+-------------+---------+
3 rows in set (0.00 sec)
```

25. Find number of customers handled by each employee.

```
mysql> SELECT salesRepEmployeeNumber, COUNT(*)
    -> FROM customers GROUP BY salesRepEmployeeNumber;
+------------------------+----------+
| salesRepEmployeeNumber | COUNT(*) |
+------------------------+----------+
|                   1002 |        2 |
|                   1003 |        1 |
+------------------------+----------+
2 rows in set (0.00 sec)
```

26. Find customers with more than 5 orders.

27. List products with total quantity ordered > 100.

```
mysql> SELECT productCode
    -> FROM orderdetails GROUP BY productCode HAVING SUM(quantityOrdered) >
100;
Empty set (0.00 sec)
```

28. Find employees managing more than 3 customers.

```
mysql> SELECT salesRepEmployeeNumber
    -> FROM customers GROUP BY salesRepEmployeeNumber HAVING COUNT(*) > 3;
Empty set (0.00 sec)
```

29. Show countries with more than 10 customers.

```
mysql> SELECT country
    -> FROM customers GROUP BY country HAVING COUNT(*) > 10;
Empty set (0.00 sec)
```

30. Find product lines having more than 5 products.

```
mysql> SELECT productLine
    -> FROM products GROUP BY productLine HAVING COUNT(*) > 5;
Empty set (0.00 sec)
```

31. List top 5 customers by total payment amount.

```
mysql> SELECT customerNumber, SUM(amount) AS total
    -> FROM payments GROUP BY customerNumber ORDER BY total DESC LIMIT 5;
+----------------+--------+
| customerNumber | total  |
+----------------+--------+
|           2002 | 500.00 |
|           2001 | 300.00 |
|           2003 | 150.00 |
+----------------+--------+
3 rows in set (0.00 sec)
```

32. Show products ordered by total sales descending.

```
mysql> SELECT productCode, SUM(quantityOrdered * priceEach) AS totalSales
    -> FROM orderdetails GROUP BY productCode ORDER BY totalSales DESC;
+-------------+------------+
| productCode | totalSales |
+-------------+------------+
| P101        |     390.00 |
| P102        |     196.00 |
| P201        |      75.00 |
+-------------+------------+
3 rows in set (0.00 sec)
```

33. List employees sorted by number of customers handled.

```
mysql> SELECT salesRepEmployeeNumber, COUNT(*) AS totalCustomers
    -> FROM customers GROUP BY salesRepEmployeeNumber ORDER BY totalCustomer
s DESC;
+-----------------------+----------------+
| salesRepEmployeeNumber | totalCustomers |
+-----------------------+----------------+
|                  1002 |              2 |
|                  1003 |              1 |
+-----------------------+----------------+
2 rows in set (0.00 sec)
```

34. Show countries sorted by total customers.

```
mysql> SELECT country, COUNT(*) AS totalCustomers
    -> FROM customers GROUP BY country ORDER BY totalCustomers DESC;
+---------+----------------+
| country | totalCustomers |
+---------+----------------+
| UK      |              1 |
| India   |              1 |
| USA     |              1 |
+---------+----------------+
3 rows in set (0.00 sec)
```

35. Display top 10 most ordered products.

```
mysql> SELECT productCode, SUM(quantityOrdered) AS total
    -> FROM orderdetails GROUP BY productCode ORDER BY total DESC LIMIT 10;
+-------------+-------+
| productCode | total |
+-------------+-------+
| P101        |    13 |
| P102        |     7 |
| P201        |     5 |
+-------------+-------+
3 rows in set (0.00 sec)
```

Multi-table JOINS

36. Get customer name, order number, and product name.

```
mysql> SELECT c.customerName, o.orderNumber, p.productName
    -> FROM customers c
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> JOIN products p ON od.productCode = p.productCode;
+--------------+-------------+--------------+
| customerName | orderNumber | productName  |
+--------------+-------------+--------------+
| ABC Corp     |        3001 | Toyota Camry |
| Global Inc   |        3003 | Toyota Camry |
| XYZ Ltd      |        3002 | Honda Civic  |
| ABC Corp     |        3001 | Yamaha Bike  |
+--------------+-------------+--------------+
4 rows in set (0.00 sec)
```

37. Show employee name, customer name, and total payments.

```
mysql> SELECT e.firstName, c.customerName, SUM(p.amount)
    -> FROM employees e
    -> JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> JOIN payments p ON c.customerNumber = p.customerNumber
    -> GROUP BY e.employeeNumber, c.customerNumber;
+-----------+-------------+---------------+
| firstName | customerName | SUM(p.amount) |
+-----------+-------------+---------------+
| Jane      | ABC Corp    |        300.00 |
| Jane      | Global Inc  |        150.00 |
| Mike      | XYZ Ltd     |        500.00 |
+-----------+-------------+---------------+
3 rows in set (0.00 sec)
```

38. List order details with product line and quantity ordered.

```
mysql> SELECT od.orderNumber, pl.productLine, od.quantityOrdered
    -> FROM orderdetails od
    -> JOIN products p ON od.productCode = p.productCode
    -> JOIN productlines pl ON p.productLine = pl.productLine;
+-------------+-------------+-----------------+
| orderNumber | productLine | quantityOrdered |
+-------------+-------------+-----------------+
|        3001 | Cars        |              10 |
|        3001 | Bikes       |               5 |
|        3002 | Cars        |               7 |
|        3003 | Cars        |               3 |
+-------------+-------------+-----------------+
4 rows in set (0.00 sec)
```

39. Display office city with employee count.

```
mysql> SELECT o.city, COUNT(e.employeeNumber)
    -> FROM offices o
    -> JOIN employees e ON o.officeCode = e.officeCode
    -> GROUP BY o.city;
+----------+------------------------+
| city     | COUNT(e.employeeNumber) |
+----------+------------------------+
| London   |                      1 |
| New York |                      2 |
+----------+------------------------+
2 rows in set (0.00 sec)
```

40. Show product vendor with total quantity sold.

```
mysql> SELECT productVendor, SUM(quantityOrdered)
    -> FROM products p
    -> JOIN orderdetails od ON p.productCode = od.productCode
    -> GROUP BY productVendor;
+---------------+---------------------+
| productVendor | SUM(quantityOrdered) |
+---------------+---------------------+
| Honda         |                   7 |
| Toyota        |                  13 |
| Yamaha        |                   5 |
+---------------+---------------------+
3 rows in set (0.00 sec)
```

Advanced Aggregations

41. Find average order value per customer.

```
mysql> SELECT customerNumber, AVG(quantityOrdered * priceEach)
    -> FROM orders o
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY customerNumber;
+----------------+-----------------------------------+
| customerNumber | AVG(quantityOrdered * priceEach) |
+----------------+-----------------------------------+
|           2001 |                        187.500000 |
|           2002 |                        196.000000 |
|           2003 |                         90.000000 |
+----------------+-----------------------------------+
3 rows in set (0.00 sec)
```

42. Show total revenue generated per country.

```
mysql> SELECT c.country, SUM(od.quantityOrdered * od.priceEach)
    -> FROM customers c
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY c.country;
+---------+----------------------------------------+
| country | SUM(od.quantityOrdered * od.priceEach) |
+---------+----------------------------------------+
| India   |                                  90.00 |
| UK      |                                 196.00 |
| USA     |                                 375.00 |
+---------+----------------------------------------+
3 rows in set (0.00 sec)
```

43. Find customers whose total payments exceed 50,000.

```
mysql> SELECT customerNumber
    -> FROM payments GROUP BY customerNumber HAVING SUM(amount) > 50000;
Empty set (0.00 sec)
```

44. List products where average order quantity > 50.

45. Find top 3 product lines by total revenue.

```
mysql> SELECT pl.productLine, SUM(od.quantityOrdered * od.priceEach)
    -> FROM productlines pl
    -> JOIN products p ON pl.productLine = p.productLine
    -> JOIN orderdetails od ON p.productCode = od.productCode
    -> GROUP BY pl.productLine ORDER BY 2 DESC LIMIT 3;
+--------------+-----------------------------------------+
| productLine | SUM(od.quantityOrdered * od.priceEach) |
+--------------+-----------------------------------------+
| Cars         |                                  586.00 |
| Bikes        |                                   75.00 |
+--------------+-----------------------------------------+
2 rows in set (0.00 sec)
```

46. Show customers with credit limit > average credit limit.

```
mysql> SELECT * FROM customers
    -> WHERE creditLimit > (SELECT AVG(creditLimit) FROM customers);
+----------------+--------------+-----------------+-----------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
| customerNumber | customerName | contactLastName | contactFirstName | phone      | addressLine1 | addressLine2 | city   | state | postalCode | country | sa
lesRepEmployeeNumber | creditLimit |
+----------------+--------------+-----------------+-----------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
|           2002 | XYZ Ltd      | Kim             | Pat             | 2222222222 | Street 2     | NULL         | London | NULL  | NW1        | UK      |
                1003 |    70000.00 |
+----------------+--------------+-----------------+-----------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
1 row in set (0.00 sec)
```

47. Find orders where total amount > 10,000.

```
mysql> SELECT orderNumber
    -> FROM orderdetails
    -> GROUP BY orderNumber HAVING SUM(quantityOrdered * priceEach) > 10000;
Empty set (0.00 sec)
```

48. List employees working in offices with more than 5 employees.

```
mysql> SELECT officeCode
    -> FROM employees GROUP BY officeCode HAVING COUNT(*) > 5;
Empty set (0.00 sec)
```

49. Find products with stock less than average stock.

```
mysql> SELECT * FROM products
    -> WHERE quantityInStock < (SELECT AVG(quantityInStock) FROM products);
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
| productCode | productName  | productLine | productScale | productVendor | productDescription | quantityInStock | buyPrice | MSRP  |
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
| P101        | Toyota Camry | Cars        | 1:18         | Toyota        | Sedan model        |             100 |    20.00 | 30.00 |
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
1 row in set (0.00 sec)
```

50. Show customers who made more than 3 payments.

```
mysql> SELECT customerNumber
    -> FROM payments GROUP BY customerNumber HAVING COUNT(*) > 3;
Empty set (0.00 sec)
```

ADVANCED SQL QUESTIONS (51–80)

Complex Joins + Aggregation

51. Find total revenue generated by each customer (include customer name), sorted by highest revenue.

```
mysql> SELECT c.customerName, SUM(od.quantityOrdered * od.priceEach) AS reve
nue
    -> FROM customers c
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY c.customerNumber ORDER BY revenue DESC;
+--------------+---------+
| customerName | revenue |
+--------------+---------+
| ABC Corp     |  375.00 |
| XYZ Ltd      |  196.00 |
| Global Inc   |   90.00 |
+--------------+---------+
3 rows in set (0.00 sec)
```

52. List customers who have placed orders but made **no payments**.

```
mysql> SELECT DISTINCT c.customerName
    -> FROM customers c
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> LEFT JOIN payments p ON c.customerNumber = p.customerNumber
    -> WHERE p.customerNumber IS NULL;
Empty set (0.00 sec)
```

53. Find total number of products ordered by each customer along with total amount spent.

```
mysql> SELECT c.customerName, SUM(od.quantityOrdered),
    -> SUM(od.quantityOrdered * od.priceEach)
    -> FROM customers c
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY c.customerNumber;
+--------------+------------------------+--------------------------------
------+
| customerName | SUM(od.quantityOrdered) | SUM(od.quantityOrdered * od.price
Each) |
+--------------+------------------------+--------------------------------
------+
| ABC Corp     |                     15 |                              3
75.00 |
| XYZ Ltd      |                      7 |                              1
96.00 |
| Global Inc   |                      3 |
90.00 |
+--------------+------------------------+--------------------------------
```

54. Show product-wise revenue and total quantity sold, sorted by revenue descending.

```
mysql> SELECT productCode, SUM(quantityOrdered),
    -> SUM(quantityOrdered * priceEach)
    -> FROM orderdetails GROUP BY productCode ORDER BY 3 DESC;
+-------------+----------------------+-----------------------------------+
| productCode | SUM(quantityOrdered) | SUM(quantityOrdered * priceEach) |
+-------------+----------------------+-----------------------------------+
| P101        |                   13 |                            390.00 |
| P102        |                    7 |                            196.00 |
| P201        |                    5 |                             75.00 |
+-------------+----------------------+-----------------------------------+
3 rows in set (0.00 sec)
```

55. Find employees along with total sales (revenue) generated from their customers.

```
mysql> SELECT e.employeeNumber, SUM(od.quantityOrdered * od.priceEach)
    -> FROM employees e
    -> JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY e.employeeNumber;
+----------------+----------------------------------------+
| employeeNumber | SUM(od.quantityOrdered * od.priceEach) |
+----------------+----------------------------------------+
|           1002 |                                 465.00 |
|           1003 |                                 196.00 |
+----------------+----------------------------------------+
2 rows in set (0.00 sec)
```

GROUP BY + HAVING (Advanced)

56. Find customers whose total purchase amount is greater than the **average purchase amount of all customers**.

57. List products that have been ordered in **more than 2 different orders**.

```
mysql> SELECT productCode
    -> FROM orderdetails GROUP BY productCode HAVING COUNT(DISTINCT orderNum
ber) > 2;
Empty set (0.00 sec)
```


```
mysql> SELECT productCode
    -> FROM orderdetails GROUP BY productCode HAVING COUNT(DISTINCT orderNum
ber) > 2;
Empty set (0.00 sec)
```


58. Find customers who ordered **more than 3 different products**.

```
mysql> SELECT customerNumber
    -> FROM orders o
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY customerNumber HAVING COUNT(DISTINCT productCode) > 3;
Empty set (0.00 sec)
```

59. Show product lines where total revenue exceeds 500.

```
mysql> SELECT p.productLine, SUM(od.quantityOrdered * od.priceEach)
    -> FROM products p
    -> JOIN orderdetails od ON p.productCode = od.productCode
    -> GROUP BY p.productLine HAVING SUM(od.quantityOrdered * od.priceEach)
> 500;
+-------------+----------------------------------------+
| productLine | SUM(od.quantityOrdered * od.priceEach) |
+-------------+----------------------------------------+
| Cars        |                                 586.00 |
+-------------+----------------------------------------+
1 row in set (0.00 sec)
```

60. Find countries where average customer credit limit is greater than 60,000.

```
mysql> SELECT country
    -> FROM customers GROUP BY country HAVING AVG(creditLimit) > 60000;
+---------+
| country |
+---------+
| UK      |
+---------+
1 row in set (0.00 sec)
```

Multi-level Joins

61. Show customer name, order number, product name, and quantity ordered for all orders.

```
mysql> SELECT c.customerName, o.orderNumber, p.productName, od.quantityOrder
ed
    -> FROM customers c
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> JOIN products p ON od.productCode = p.productCode;
+--------------+-------------+--------------+-----------------+
| customerName | orderNumber | productName  | quantityOrdered |
+--------------+-------------+--------------+-----------------+
| ABC Corp     |        3001 | Toyota Camry |              10 |
| ABC Corp     |        3001 | Yamaha Bike  |               5 |
| XYZ Ltd      |        3002 | Honda Civic  |               7 |
| Global Inc   |        3003 | Toyota Camry |               3 |
+--------------+-------------+--------------+-----------------+
4 rows in set (0.00 sec)
```

62. Find employee name, office city, and number of customers handled.

```
mysql> SELECT e.firstName, o.city, COUNT(c.customerNumber)
    -> FROM employees e
    -> JOIN offices o ON e.officeCode = o.officeCode
    -> LEFT JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> GROUP BY e.employeeNumber;
+-----------+----------+-----------------------+
| firstName | city     | COUNT(c.customerNumber) |
+-----------+----------+-----------------------+
| John      | New York |                     0 |
| Jane      | New York |                     2 |
| Mike      | London   |                     1 |
+-----------+----------+-----------------------+
3 rows in set (0.00 sec)
```

63. Display product line, product name, and total revenue generated.

```
mysql> SELECT pl.productLine, p.productName,
    -> SUM(od.quantityOrdered * od.priceEach)
    -> FROM productlines pl
    -> JOIN products p ON pl.productLine = p.productLine
    -> JOIN orderdetails od ON p.productCode = od.productCode
    -> GROUP BY p.productCode;
+-------------+--------------+----------------------------------------+
| productLine | productName  | SUM(od.quantityOrdered * od.priceEach) |
+-------------+--------------+----------------------------------------+
| Cars        | Toyota Camry |                                 390.00 |
| Cars        | Honda Civic  |                                 196.00 |
| Bikes       | Yamaha Bike  |                                  75.00 |
+-------------+--------------+----------------------------------------+
3 rows in set (0.00 sec)
```

64. Show order number, customer name, and total order value.

```
mysql> SELECT o.orderNumber, c.customerName,
    -> SUM(od.quantityOrdered * od.priceEach)
    -> FROM orders o
    -> JOIN customers c ON o.customerNumber = c.customerNumber
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY o.orderNumber;
+-------------+--------------+----------------------------------------+
| orderNumber | customerName | SUM(od.quantityOrdered * od.priceEach) |
+-------------+--------------+----------------------------------------+
|        3001 | ABC Corp     |                                 375.00 |
|        3002 | XYZ Ltd      |                                 196.00 |
|        3003 | Global Inc   |                                  90.00 |
+-------------+--------------+----------------------------------------+
3 rows in set (0.00 sec)
```

65. List employees whose customers have placed more than 5 orders in total.

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual t
hat corresponds to your MySQL server version for the right syntax to use ne
r '65' at line 1
mysql> SELECT e.employeeNumber
    -> FROM employees e
    -> JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> GROUP BY e.employeeNumber HAVING COUNT(o.orderNumber) > 5;
Empty set (0.00 sec)
```

Aggregation with Filtering

66. Find customers whose total payments are less than their total order value.

```
mysql> SELECT e.employeeNumber
    -> FROM employees e
    -> JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> GROUP BY e.employeeNumber HAVING COUNT(o.orderNumber) > 5;
Empty set (0.00 sec)
```

67. List products where total quantity sold is less than average quantity sold across all products.

```
mysql> SELECT productCode
    -> FROM orderdetails
    -> GROUP BY productCode
    -> HAVING SUM(quantityOrdered) < (SELECT AVG(total)
    -> FROM (SELECT SUM(quantityOrdered) AS total FROM orderdetails GROUP BY
 productCode) t);
+-------------+
| productCode |
+-------------+
| P102        |
| P201        |
+-------------+
2 rows in set (0.00 sec)
```

68. Find customers who have not placed any orders but have made payments.

```
mysql> -- 68;
mysql> SELECT c.customerNumber
    -> FROM customers c
    -> LEFT JOIN orders o ON c.customerNumber = o.customerNumber
    -> JOIN payments p ON c.customerNumber = p.customerNumber
    -> WHERE o.customerNumber IS NULL;
Empty set (0.00 sec)
```

69. Show employees who have not been assigned any customers.

```
mysql> SELECT e.employeeNumber
    -> FROM employees e
    -> LEFT JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> WHERE c.customerNumber IS NULL;
+----------------+
| employeeNumber |
+----------------+
|           1001 |
+----------------+
1 row in set (0.00 sec)
```

70. Find orders that contain more than 1 product (multi-line orders).

```
mysql> -- 70;
mysql> SELECT orderNumber
    -> FROM orderdetails GROUP BY orderNumber HAVING COUNT(*) > 1;
+--------------+
| orderNumber |
+--------------+
|        3001 |
+--------------+
1 row in set (0.00 sec)
```

71. Find top 3 customers based on total order value.

```
mysql> SELECT customerNumber, SUM(quantityOrdered * priceEach) AS total
    -> FROM orders o
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY customerNumber ORDER BY total DESC LIMIT 3;
+----------------+--------+
| customerNumber | total  |
+----------------+--------+
|           2001 | 375.00 |
|           2002 | 196.00 |
|           2003 |  90.00 |
+----------------+--------+
3 rows in set (0.00 sec)
```

72. Show top 5 products based on total revenue.

```
mysql> SELECT productCode, SUM(quantityOrdered * priceEach) AS revenue
    -> FROM orderdetails GROUP BY productCode ORDER BY revenue DESC LIMIT 5;
+-------------+---------+
| productCode | revenue |
+-------------+---------+
| P101        |  390.00 |
| P102        |  196.00 |
| P201        |   75.00 |
+-------------+---------+
3 rows in set (0.00 sec)
```

73. List top 2 employees based on sales generated.

```
mysql> SELECT e.employeeNumber, SUM(od.quantityOrdered * od.priceEach) AS sa
les
    -> FROM employees e
    -> JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY e.employeeNumber ORDER BY sales DESC LIMIT 2;
+----------------+--------+
| employeeNumber | sales  |
+----------------+--------+
|           1002 | 465.00 |
|           1003 | 196.00 |
+----------------+--------+
2 rows in set (0.00 sec)
```

74. Find top 3 countries by number of customers.

```
mysql> SELECT country, COUNT(*) AS total
    -> FROM customers GROUP BY country ORDER BY total DESC LIMIT 3;
+---------+-------+
| country | total |
+---------+-------+
| UK      |     1 |
| India   |     1 |
| USA     |     1 |
+---------+-------+
3 rows in set (0.00 sec)
```

75. Show top 5 most expensive products based on MSRP.

```
mysql> SELECT * FROM products ORDER BY MSRP DESC LIMIT 5;
+-------------+--------------+-------------+--------------+---------------+-
------------------+-----------------+----------+-------+
| productCode | productName  | productLine | productScale | productVendor |
productDescription | quantityInStock | buyPrice | MSRP  |
+-------------+--------------+-------------+--------------+---------------+-
------------------+-----------------+----------+-------+
| P101        | Toyota Camry | Cars        | 1:18         | Toyota        |
Sedan model        |             100 |    20.00 | 30.00 |
| P102        | Honda Civic  | Cars        | 1:18         | Honda         |
Compact model      |             150 |    18.00 | 28.00 |
| P201        | Yamaha Bike  | Bikes       | 1:12         | Yamaha        |
Sport bike         |             200 |    10.00 | 15.00 |
+-------------+--------------+-------------+--------------+---------------+-
------------------+-----------------+----------+-------+
3 rows in set (0.00 sec)
```

76. Find average order value per customer and list only those above overall average.

```
mysql> SELECT customerNumber, AVG(quantityOrdered * priceEach) AS avgVal
    -> FROM orders o
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY customerNumber
    -> HAVING avgVal > (SELECT AVG(quantityOrdered * priceEach) FROM orderde
tails);
+----------------+------------+
| customerNumber | avgVal     |
+----------------+------------+
|           2001 | 187.500000 |
|           2002 | 196.000000 |
+----------------+------------+
2 rows in set (0.00 sec)
```

77. Show product-wise average price sold (priceEach) vs MSRP difference.

78. Find percentage contribution of each product to total revenue.

```
mysql> SELECT productCode,
    -> SUM(quantityOrdered * priceEach) /
    -> (SELECT SUM(quantityOrdered * priceEach) FROM orderdetails) * 100 AS
percentage
    -> FROM orderdetails GROUP BY productCode;
+-------------+------------+
| productCode | percentage |
+-------------+------------+
| P101        |  59.001513 |
| P102        |  29.652042 |
| P201        |  11.346445 |
+-------------+------------+
3 rows in set (0.00 sec)
```

79. Calculate average quantity ordered per order for each customer.

```
mysql> SELECT customerNumber, AVG(quantityOrdered)
    -> FROM orders o
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY customerNumber;
+----------------+----------------------+
| customerNumber | AVG(quantityOrdered) |
+----------------+----------------------+
|           2001 |               7.5000 |
|           2002 |               7.0000 |
|           2003 |               3.0000 |
+----------------+----------------------+
3 rows in set (0.00 sec)
```

80. Find customers whose **largest single order value** exceeds 10,000.

```
mysql> SELECT customerNumber
    -> FROM orders o
    -> JOIN orderdetails od ON o.orderNumber = od.orderNumber
    -> GROUP BY customerNumber
    -> HAVING MAX(quantityOrdered * priceEach) > 10000;
Empty set (0.00 sec)
```